TEKNOFEST

HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

UÇAN ARABA YARIŞMASI

FİNAL TASARIM RAPORU

TAKIM ADI: KARAYEL

TAKIM ÜYELERİ:

Gizem BEDENLİ

Tolunay DAĞ

Seda Nur ÖZSUNAR

Orhan HIZALAN

Abdulrahman Salah Ahmed AL-BANDAR

DANIŞMAN ADI: Tarık ÜNLER

İçindekiler Tablosu

1. SENARYOLARDA KULLANILAN SENSÖRLER VE PROGRAMLAMA DİLİ 4
1.1. Kullanılan Programlama Dili ve Özellikleri 4
1.2. Tercih Edilebilecek Diğer Dil 4
1.3. Barometre Sensörü Çalışma Prensibi ve GNSS ile Barometre İrtifa Verileri Farkı 5
1.4. İvmeölçer ve Dönüölçer Sensörlerinin Genel Hata Karakteristikleri ile Özellikleri 5
2. SENARYO 1 İÇIN TASARLANAN ALGORİTMA VE ÖZELLİKLERİ 9
2.1. Hava Aracının Gitmesi Gereken Yönün Tespiti İçin Hesaplanan Algoritma 9
2.2. Hava Aracının Gitmesi Gereken Koordinatların Veri Yapısı 11
2.3. İniş için Gerekli Koşul Seçimleri 11
2.4. İnişte İrtifa Tespiti için Kullanılan Sensör 12
2.5. İniş Esnasında Yere Değme Hızı 13
3. SENARYO 4: YASAK BÖLGEDEN KAÇIŞ İÇİN TASARLANAN ALGORİTMA ... 14
3.1. Yasak Bölge Tespitinde Kullanılan Veri Yapısı 14
3.2. Yasak Bölgelere Girmemek için Tasarlanan Algoritma 14
4. SENARYO 7: TRAFİKTE OLAN DİĞER UÇAN ARABALARI GÖZETEREK KAZA YAPMADAN UÇMAYI SAĞLAYAN ALGORİTMA 16
4.1. Diğer Hava Araçlarının Konumlarını Tespit Eden Veri Yapısı 16
4.2. Hava Araçları ile Kaza Yapmamak Için Tasarlanan Algoritma 16
5. SENARYO 8: EN YAKIN HASTANEYİ BULMANIZI SAĞLAYAN ALGORİTMA 17
5.1. Hastanelerin Olduğu Veri Yapısı 17
5.2. En Yakin Hastanene Seçiminde Kullanılan Algoritma 17
6. SENARYO 10: GNSS ARIZASI YAŞANDIĞINDA İNİŞİ TAMAMLAYAN ALGORİTMA 18
6.1. GNSS Arızasının Tespit Edildiği Veri Yapısı 18
6.2. GNSS Arıza Sonrası Hava Aracının Irtifasının Hesapladığı Algoritma 18
7. SENARYO 11: GNSS ARIZASI YAŞANDIĞINDA SEYİR YAPMAYA DEVAM EDEN VE GÖREVİ TAMAMLAMAYI SAĞLAYAN ALGORİTMA 19

7.1. GNSS Arızası Yaşandıktan Sonra Hava Aracının Pozisyonunun(Yatay) Hesaplandığı Algoritma 19

8. SENARYO 12 : GNSS ARIZA DURUMUNU TESPIT EDEN ALGORITMA 20

8.1. GNSS Karıştırma (Jamming) Ve Yanıltma (Spoofing) Açıklaması Ve Farkları 20

8.2. GNSS Yanıltma (Spoofing) Durumunu Tespit Eden Algoritma 21

9. SENARYO 13: İNİŞİ TAMAMLAMAK İÇİN KULLANILAN SENSÖR SEÇİMİN YAPILAN ALGORİTMA 21

9.1. İniş Için Seçilen Irtifa Kaynağı Sensörü 21

9.2.Diğer Sensörlerin Seçilmeme Sebepleri 22

9.3. Bu Seçimi Gerçekleştiren Algoritma 22

10. SENARYO 15: MANYETOMETRE VERİ KAYBI YAŞANDIĞINDA GİDİLMESİ GEREKEN HEDEFE HAVA ARACININ YÖNELMESİNİ SAĞLAYAN ALGORİTMA.. 22

10.1. Manyetometre Veri Kaybının Tespiti 22

10.2. Hava Aracının Gitmesi Gereken Yönü Hesaplayan Algoritma 23

10.3. Hava Aracının Gittiği Yönü Hesaplayan Algoritma 23

11. SENARYO 16: ŞARJ DURUMUNU GÖZETEREK ŞARJ İSTASYONLARİNİ TESPİT EDEN VE HAVA ARACINI ORAYA YÖNLENDİREN ALGORİTMA 23

11.1. Batarya durumları için kullanılan veri yapısı 23

11.2. Şarj İstasyonuna İnilmesi Gereken Batarya Gerilim Eşiği ve Nedenleri 24

11.3. Şarj Istasyonlarının Olduğu Veri Yapısı 24

12. KAYNAKÇA 25

1. SENARYOLARDA KULLANILAN SENSÖRLER VE PROGRAMLAMA DİLİ

1.1. Kullanılan Programlama Dili ve Özellikleri

Yapılan uçan araba projesi için Python programlama dilinin kullanılmasına karar kılındı. Bu seçimin birden çok sebebi vardır. Öncelikle piyasada yer alan programlama dilleri arasında en esnek, en güçlü ve en basitlerinden birisi olup aynı zamanda zengin modülleri ve kütüphanesi sayesinde verimli bir çalışma ortamı sunar.

Tercih edilmesine diğer sebep ise sağladığı yüksek üretkenlik seviyesidir. İşlerini kısa sürede gerçekleştirmek isteyen kullanıcılar için en uygun programdır. Kapsamlı bir program olmasına rağmen çok kolay okunan bir dile sahiptir.

Şu an gerçekleştirilen çalışmada yapay zekadan faydalanılmakta ve yazılmakta olan kodlarla tahminler yapılarak potansiyel cevaplar ortaya çıkarılmaktadır. Bunun sayesinde projeyi gerçek hayata geçirmeden önce sanal ortamda testler yapılmaktadır. Bu testlerle birlikte ortaya çıkabilecek olası hatalar farkedilip önceden önlem alınmış olur.

Diğer bir avantajı ise Phyton programının açık kaynaklı bir yazılım olup yüksek verimliliğe ücretsiz erişim sağlamasıdır.

Yukarıda sayılan bu sebepler bu programın tercih edilmesine öncülük etmiştir.[1–3]

1.2. Tercih Edilebilecek Diğer Dil

Üzerinde çalışılan proje için Java programlama dili tercih edilebilirdi. Java, yüksek geliştirici verimliliği ile güçlü, zengin özellikli dil olması özelliğiyle Python diline en çok benzeyen dildir. Çok sayıda açık kaynaklı kütüphaneye erişim imkanı vardır.

Dilin tercih edilmeme sebebi yavaş çalışan bir program olmasıdır. Ayrıca Java'nın ayrıntı düzeyinin yüksek olması pozitif bir etki olarak görülmesine rağmen, kodu oldukça karmaşık hale getirir. Bu da kodun okunmasını zorlaştırır.[1–3]

1.3. Barometre Sensörü Çalışma Prensibi ve GNSS ile Barometre İrtifa Verileri Farkı

Uçan arabanın irtifasını ve basıncını ölçmek için kullanılan barometre sensörü, otopilot işlevinin yerine getirilmesi için elzem bir yapıdır. Aneroid barometreler küçük, hafif ve taşınabilir olduklarından dolayı uçaktaki rakım ölçümleri için tercih edilen bir araçtır. Barometrenin içi küçük, esnek metal kutu şeklindedir. Bu kutudan hava pompalandığı için dış hava basıncındaki değişimler bu esnek kutunun genleşme-büzülme hareketlerini gerçekleştirmesini sağlar. Bu hareketler, barometrede bulunan iğneyi hareket ettiren mekanik kollarını da harekete geçirir. İğnenin barometre ekranında hareket etmesi ile basınç değişimleri ekrandan kolaylıkla okunur.[4–7]

Global Navigation Satellite Systems (GNSS) olarak bilinen Türkçe karşılığı 'Küresel Navigasyon Uydu Sistemi' olan sistem, uzaydan sinyaller yollayabilen ve aracın dünyadaki coğrafi konumunu belirlemek için küçük uyduları kullanır.

GNSS sistemlerinden GPS'i baz alarak barometre ile kıyaslama yapılırsa;

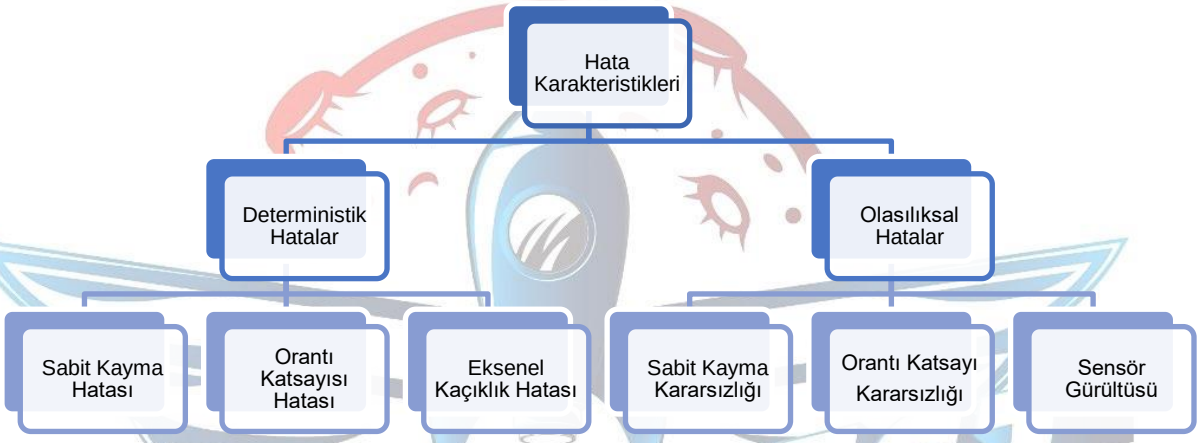
GPS alıcısı belirli bir zamanda uyduların nerede olduklarını bilir ve trilaterasyon yöntemini kullanarak 3 veya daha fazla uydu ile iletişime geçerek kendi yerini belirler. Dünyanın şeklinin elips olmasından dolayı uydularla olan mesafe hesaplamalarında olası matematiksel hatalar ortaya çıkar. Barometre ile yükseklik ölçüldüğünde ise bulunulan yüksekliği tespit etmek için atmosferik basınç kullanır. Atmosferik basınç ile deniz seviyesinden (Deniz seviyesi referans noktasıdır.) ne kadar yüksekte olduğu belirlenir. Buradan bulunan irtifaya ve GPS yönteminden bulunan irtifaya bakıldığında barometre ile bulunan yüksekliğin daha doğru sonuçlar verdiği görülür.[8–10]

1.4. İvmeölçer ve Dönüölçer Sensörlerinin Genel Hata Karakteristikleri ile Özellikleri

[11]Üç eksen boyunca yerleştirilen ivmeölçer hız ve konum bilgisini, dönüölçer ise hangi tarafa yönelim gösterdiğinin bilgisini sağlar. Çalışma prensipleri gereğince, sensör kaynaklı hata ve

gürültülerin zamanla artmasına sebep olmaktadır. Bu sebeple uzun süreli kullanımlarda tatmin edici sonuçlar ortaya çıkmaz. Hataları belirlemek için sensörlere bilinen giriş değerleri uygulanır ve elde edilen çıkış verileri incelenir.

İvmeölçer ve dönüölçerde ortaya çıkan hata karakteristikleri fiziksel yapılarından dolayı ortaya çıkar. Bu hataları deterministik hatalar ve olasılıksal hatalar olmak üzere iki grupta incelenir.



Şekil 1: Hata Karakteristikleri

Aşağıda verilen tablolarda ivmeölçer ve dönüölçerden kaynaklanan hatalar açıklanmıştır.

Tablo 1: İvmeölçer Hata Kaynakları

İvmeölçerden Kaynaklanan Hatalar	
İvmeölçer Sabit Kayma Hatası	Sensöre ivme uygulanmadığı durumda ölçülen değerdir. Bu hata terimini sıcaklıkla uyarılan değişimler veya tekrarlanabilirlik gibi parametreler etkileyebilir.
İvmeölçer Orantı Katsayısı Hatası	Sensörün çıkışında meydana gelen değişimle girişindeki değişimin oranı bu hatayı gösterir. İvmeölçer etkin olarak kullanıldığı sürece bu hata da olacaktır.
İvmeölçer Eksenel Kaçıklık Hatası	İvmeölçerin dik monte edilmediği durumlarda meydana gelir. Bu hatayı belirlemek diğer hata türlerine göre daha kolaydır. 3 eksen etrafında pozitif ve negatif yönde olmak üzere statik kalibrasyon testi gerçekleştirilerek hata tespiti yapılır.
İvmeölçer Sabit Kayma Kararsızlığı	Bu hata, çalışma sırasında meydana gelen değişimlerden kaynaklı hataları içerir. İvmeölçer sabit kayma kararsızlığı $\frac{g}{\sqrt{Hz}}$ olarak ifade edilir.
İvmeölçer Orantı Katsayısı Kararsızlığı	Orantı katsayısının zamanla değişimi şeklinde açıklanabilir. İvmeölçerin çalıştığı her an bu hata var olur.
İvmeölçer Rastgele Sensör Gürültüsü	Yüksek frekansa sahip gürültüler rastgele sensör gürültüsünü oluşturur.

TEKNOFEST İSTANBUL HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

Tablo 2 : Dönüölçer Hata Kaynakları

Dönüölçerden Kaynaklanan Hatalar	
Dönüölçer Sabit Kayma Hatası	Belirli bir süre boyunca ölçülen dönü değerlerinin ortalaması alınarak dönüölçer sabit kayma hatasına ulaşılır. Bu aşamada dönüölçere herhangi bir giriş uygulanmaz.
Dönüölçer Sabit Kayma Kararsızlığı	Çalışma sırasında meydana gelen değişimler bu hatanın oluşmasına sebebiyet verir.
Dönüölçer Orantı Katsayısı Hatası	Dönüölçer orantı katsayısı, açısal hız girişi uygulanan eksen boyunca dönüölçer hassasiyetini oluşturur ve bu hata oluşur.
Dönüölçer Orantı Katsayısı Kararsızlığı	Dönüölçer orantı katsayısı kararsızlığı, orantı katsayısındaki zamanla değişimidir. Bu hatanın çok uzun süre gerektiren dinamik testlerin sonucu anlaşılır.
Dönüölçer Eksenel Kaçıklık Hatası	Dönüölçerlerin yanlış hizalandığı zaman dönüölçerler birbirine dik konumda olmayabilir. Bunun sonucunda her eksenin diğer iki eksen etrafındaki dönüye karşı hassasiyet meydana gelir ve hata oluşur. Bu hatayı belirlemek için 3 eksen etrafında pozitif ve negatif yönde olmak üzere dinamik kalibrasyon testi uygulanır.

TEKNOFEST İSTANBUL HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

2. SENARYO 1: HAVA ARACININ B NOKTASINA GİDİŞİ İÇİN TASARLANAN ALGORİTMA VE ÖZELLİKLERİ

2.1. Hava Aracının Gitmesi Gereken Yönü Tespit Etmek İçin Hesaplanan Algoritma

ALGORİTMA	AÇIKLAMA
irtifada = False	Aracın gereken irtifaya ulaşmadığını gösterir.
bitti = False	Hedef noktaya yönelimin henüz bulunmadığını gösterir.
inis = False:	Aracın henüz hedef noktaya (B noktasına) inişinin gerçekleşmediğini gösterir.
faktor=0.1	Girilen hız değerini küçültmek için atanmış bir değerdir.
varis= cezeri.hedefler[0] enlem= cezeri.gnss.enlem boylam= cezeri.gnss.boylam	Varışın (B noktasının) koordinatları ‘varis’ değişkenine, aracın anlık koordinatları’enlem’ ve ‘boylam’ değişkenine atanmıştır.
aci = math.atan2(varis.boylam - boylam ,varis.enlem - enlem)*(180/math.pi)	Aracın hedef noktayla arasındaki açısı ‘aci’ değişkenine atanmıştır.
yonelim = cezeri.manyetmetre.veri*(180/math.pi)	Aracın yöneliminin derece cinsinden değeri ‘yonelim’ değişkenine atanmıştır.

def rota_takip(A_j, A_i, B_j, B_i):
cezeri.ileri_git(cezeri.hizli)
aci = math.atan2(B_j - A_j ,B_i - A_i)*(180/math.pi)
yonelim = cezeri.manyetometre.veri*(180/math.pi)
if 5 > aci - yonelim > 1:
cezeri.don(hiz*faktor)
else:
cezeri.don(-1*hiz*faktor)
	Araç hedef noktası (B_i ve B_j) ile bulunduğu konum(A_i ve A_j) arasındaki açıyı hesaplayarak hedef noktasına doğru girilen hız değeriyle yönelim yapmaktadır. Bu durum bir fonksiyon içine atanmıştır.

if cezeri.barometre.yukseklik > cezeri.irtifa_araligi[0] and not bitti:
if irtifada == False:
irtifada == True
cezeri.dur()
cezeri.don(hiz)
if abs(aci - yonelim) < 5:
cezeri.dur()
bitti = True
elif not bitti:
cezeri.yukari_git(cezeri.hizli)
	Araç mininum irtifaya ulaşmak için yükselirken aynı zamanda hedef nokta yönelimini bulmaktadır.

if bitti and not inis: rota_takip(boylam, enlem, varis.boylam, varis.enlem	Araç yönelimini tamamladığında yukarıda gösterilen fonksiyon kullanılarak hedef noktaya doğru yol almaktadır.

if math.sqrt((boylam - varis.boylam)**2 + (enlem - varis.enlem)**2) < 0.5:
if inis == False:
cezeri.dur()
inis = True
	Araç hedef noktasıyla arasındaki uzaklığı hesaplamaktadır ve 0.5 hata payıyla bu noktaya ulaştığında iniş yapmaktadır.

if(not math.isinf(cezeri.lidar.mesafe)) and (not cezeri.lidar.hata):
if cezeri.lidar.mesafe < 30:
cezeri.asagi_git(cezeri.yavas)
elif (not math.isinf(cezeri.radar.mesafe)) and (not cezeri.radar.hata):
if cezeri.radar.mesafe < 30:
cezeri.asagi_git(cezeri.yavas)
else:
cezeri.asagi_git(cezeri.hizli)
	Lidar sensörü ile anlık irtifasını hesaplayarak hızını ayarlar. Eğer ki yer ile arasındaki mesafe 30'tan küçükse yavaş iniş gerçekleştirir.
	
	Lidar sensörü arıza yaptığında mesafeyi radar sensörü ile ölçerek iniş yapmaktadır.

2.2. Hava Aracının Gitmesi Gereken Koordinatların Veri Yapısı

'cezeri.hedefler' dizisi aracın gitmesi gereken koordinatları içeren bir dizidir. Örnek olarak ilk hedef 'cezeri.hedefler[0]' ile elde edilir.

2.3. İniş için Gerekli Koşul Seçimleri

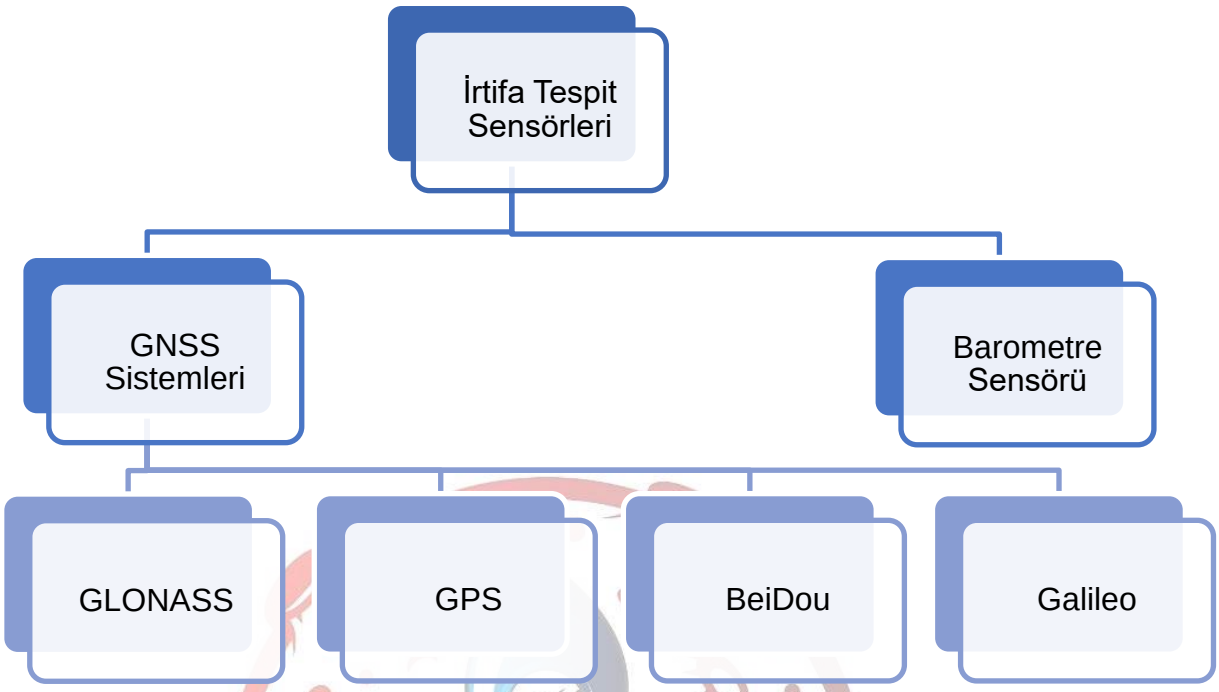
- Meteorolojik Şartlar: Aracın emniyetli bir şekilde inişe geçebilmesi için gerekli meteorolojik şartlar sağlanmalıdır. Pilotun görüş mesafesi en az 500 m olmalıdır. Bu yüzden uçuş faaliyetlerini doğrudan etkileyen rüzgâr, türbülans, sıcaklık, sis, yağışlar, don ve buzlanma gibi olaylar görüş mesafesini ve uçuş rotasının yönünü direk etkileyeceğinden dolayı uçuşlar hava koşulları uygun olduğunda gerçekleştirilmelidir.
- Apron: Uçan arabanın inişe geçerken herhangi bir canlının veya başka bir aracın zarar görmemesi için gerekli apron koşulları oluşturulur. Aracın etrafındaki nesnelerle arasındaki mesafesi en az 50 metre uzunluğunda tutulmalıdır. Pervaneli sistemlerde pervaneler özellikle motorları durduktan sonra sessizdir ve dönen pervaneler kolayca görünmez bu sebeple güvenlik için pervaneler tamamen durmadan aracın altına girilmemelidir.
- Basınç-İrtifa: Yüksek irtifalarda basınç, sıcaklık ve oksijen yoğunluğunun çok düşük seviyede olması insan sağlığı açısından olumsuz bir duruma neden olduğundan uçan araba kabini basınçlandırma yoluyla uçuş irtifasına göre daha dü-

şük bir irtifada tutulur. Uçan araba inişe geçerken irtifa kaybından kaynaklı basınç, sıcaklık ve oksijen yoğunluğunda değişim yaşanır. İklimlendirme sistemlerini baz alırsak; dağıtım, basınçlandırma, sıcaklık kontrol, ekipman soğutma, ısıtma ve soğutma gibi sistemleriyle uçan arabada kabin uygun koşullara ulaşır. İniş bu sistemlere dikkat edilerek gerçekleştirilir.

- Hava Trafiği: Havada uçuş gerçekleştiren birçok araç bulunmaktadır. Bu araçlar uçuş esnasında olası kazalardan kaçınmak için birbirlerinin konumlarına ve aralarındaki asgari mesafeye dikkat etmek zorundadırlar. Hava trafik kontrolörleri, pilotlarla telsiz bağlantısı kurarak kalkış, uçuş ve iniş için hangi rotayı kullanacaklarını bildirirler. Bu bildiriler dikkat alınarak güvenli bir uçuş gerçekleştirilmelidir.
- Sensör Çalışma Durumları: Araç iniş sırasında belirli sensörler kullanılmaktadır ve sensörlerin doğru veriler vermesi gerekmektedir. Örneğin ultrasonik sensörler, varlık, seviye, durum ve mesafe algılama gibi görevler üstlenir. Bu sensörün arıza durumunda, iniş bölgesinde herhangi bir nesne bulunuyorsa algılama gerçekleştiremez ve o nesneye temas ederek uçan arabanın hasar almasına neden olur. Bu yüzden hem iniş sırasında hem de uçuş sırasında sensör kontrolü yapılmalıdır.[12–16]

2.4. İnişte İrtifa Tespiti için Kullanılan Sensör

Sensörler, basınç, hareket, sıcaklık, ışık, darbe gibi fiziksel değişimleri algılayıp bağlı olduğu işlemciye iletir. İrtifa tespiti için kullanılan çeşitli sensörler bulunmaktadır ve bu sensörler şekil 2 ‘de gösterilmiştir.



Şekil 2: İrtifa Tespit Sensörleri

Araçların güvenli bir uçuş gerçekleştirebilmesi için doğru ve anlık irtifa tespiti yapması gerekmektedir. Bu durum için en güvenilir olan barometre sensörünün seçimi yapılmıştır. Barometre sensörleri, sıvı -gaz akışı, hız, su seviyesi ve rakım gibi diğer değişkenleri dolaylı olarak ölçmek için de kullanılabilir.

Rakım ölçme, uçaklarda, roketlerde, uydularda, hava balonlarında ve diğer hava taşıtlarında aktif olarak kullanılır. Bu sensör, irtifaya göre basınçtaki değişiklikler arasındaki ilişkiyi kullanır.

Bir barometre sensörü, dahili basınç sensörünü kullanarak mutlak hava basıncını sürekli olarak ölçer. Bu ölçüm ve referans değerlerine dayanarak, irtifa hesabı yapılır.

2.5. İniş Esnasında Yere Değme Hızı

Aracın yere değme hızı, NASA ve UBER'den alınan veriler doğrultusunda 2.5 m/s olarak belirlenmiştir.

3. SENARYO 4: YASAK BÖLGEDEN KAÇIŞ İÇİN TASARLANAN ALGORİTMA

3.1. Yasak Bölge Tespitinde Kullanılan Veri Yapısı

'cezeri.harita.ucusa_yasakli_bolgeler' list yapısı içerisinden yasaklı bölgenin kordinatları elde edilip 'yasak_bolgeye_geldi' değeri ile aracın konumunun yasaklı bölgede olup olmadığı kontrol edilir. Eğer 'yasak_bolgeye_geldi' değeri 'True' ise konumun yasaklı bölgede olduğunu ifade etmektedir.

3.2. Yasak Bölgelere Girmemek İçin Tasarlanan Algoritma

ALGORİTMA	ACIKLAMA
	Daha önce senaryo 1'in algoritmasında açıklandığı gibi araç bulunduğu konumdan kalkış yaparak hedef noktaya rota oluşturmaktadır. Not: Tekrara düşmemek için bu algoritmalar bir daha gösterilmeyecektir.
yasak_bolgeye_geldi = False	Aracın yasaklı bölgeye ulaşmadığını gösterir.
yasak_bolge_varmi = False	Hedef noktaya giderken yasaklı bölgenin olmadığını gösterir
sinira_geldi = False	Yasaklı bölgeye girmeden etrafında belirlenen sınıra ulaşmadığını gösterir.
hazir = False	Aracın hedef noktaya gitmek için hazır olmadığını gösterir.
dondu = False	Aracın A noktasından B noktasına gitmek için yöneliminin henüz tamamlanmadığını gösterir
minimum = 1000	Yasakli bölgeye olan uzaklık ilk başta bu değer olarak atanmaktadır.

def yasakli_bolge_varmi(A_j, A_i, B_j, B_i):
  y = []
  if B_i == A_i:
  egim = 0
  else:
  egim = (B_j - A_j)/(B_i - A_i)
 	
  if A_i > B_i:
  x = list(range(int(B_i), int(A_i)))
  elif A_i < B_i:
  x = list(range(int(A_i), int(B_i)))
 	
  for i in range(0, len(x)):
  y.append((x[i] - B_i)*egim + B_j)
  if cezeri.harita.bolge(y[i], x[i]).ucusa_yasakli_bolge:
  return True
	Aracın anlık aldığı enlem ve boylam değerlerinden varmak istediği konuma giderken doğrusal yörünge çizerek önünde yasaklı bölge bulunduğunu tespit eder.

for bolge in cezeri.harita.ucusa_yasakli_bolgeler: if math.sqrt((boylam - bolge.boylam)**2 + (enlem - bolge.enlem)**2) < minimum : minimum = math.sqrt((boylam - bolge.boylam)**2 + (enlem - bolge.enlem)**2)	Hedef noktaya giderken aracın yasaklı bölgeyle arasındaki uzaklığı hesaplar.

if minimum < 6 and not sinira_geldi:
  yasak_bolgeye_geldi = True
  cezeri.don(hiz*0.35)
  if minimum < 3.5:
  sinira_geldi = True
if minimum > 3.2 and sinira_geldi:
  cezeri.don(-1*hiz*0.45)
if minimum < 3.1 and sinira_geldi:
  cezeri.don(hiz*0.45)
	Araç ile yasaklı bölge arasındaki optimum değerler belirlerlenir. Belirlenen değerlere göre araç yasaklı bölgeye girmeden uzaklaşır.

if not yasakli_bolge_varmi(boylam, enlem, varis.boylam, varis.enlem) and not hazir:
  if dondu == False:
  cezeri.dur()
 	
  cezeri.don(hiz)
 	
  if abs(aci - yonelim) < 5:
  cezeri.dur()
  hazir = True
if hazir:
  rota_takip(boylam, enlem, varis.boylam, varis.enlem)
	Eğer arac hedef noktaya giderken önünde yasaklı bölge yoksa hedef noktaya ilerlemeye devam edilir.

4. SENARYO 7: TRAFİKTE OLAN DİĞER UÇAN ARABALARI GÖZETEREK KAZA YAPMADAN UÇMAYI SAĞLAYAN ALGORİTMA

4.1. Diğer Hava Araçlarının Konumlarını Tespit Eden Veri Yapısı

'cezeri.trafik' dizisi trafik bölgelerini içeren bir listedir. Bu list yapısının içinde 'trafik' sayacı döndürerek 'trafik.enlem'-'trafik.boylam' verileri ile hava aracının enlem ve boylam koordinatları elde edilir. Örnek olarak 'cezeri.trafik[0]' bize ilk trafik bölgesinin koordinatları verir.

4.2. Hava Araçları ile Kaza Yapmamak İçin Tasarlanan Algoritma

ALGORİTMA	AÇIKLAMA

for trafik in cezeri.trafik:
  if m.sqrt((boylam - trafik.boylam)**2 + (enlem - trafik.enlem)**2) < 2:
  cezeri.dur()
  else:
  rota_takip(boylam, enlem, varis_boy, varis_en)
	Trafikte bulunan araçlar 'trafik' döngüsü içerisinde kontrol edilir. Cezeri aracına herhangi bir araacın yaklaşması durumunda araç durmaktadır. Tehlike geçince rotasına devam etmektedir.

5. SENARYO 8: EN YAKIN HASTANENİN BULUNMASINI SAĞLAYAN ALGORİTMA

5.1. Hastanelerin Olduğu Veri Yapısı

'cezeri.harita.hastaneler' dizisinden haritadaki hastanelerin koordinatları list yapısı ile elde edilir. Örnek olarak 1. hastanenin koordinatlarını veren fonksiyon: 'cezeri.harita.hastaneler[0]'.

5.2. En Yakın Hastane Seçiminde Kullanılan Algoritma

ALGORİTMA	AÇIKLAMA
olculdu = False	En yakın hastane koordinatlarının henüz bulunmadığını gösterir.
minimum = 1000	Hastane uzaklığı ilk başta bu değer olarak atanmaktadır.
son_olcum = cezeri.ros_zamani()	Simülasyon boyunca geçen süreyi hesaplayan değişkendir.

if cezeri.ros_zamani() - son_olcum > 1 and not olculdu:
for i in range(0, len(cezeri.harita.hastaneler)):
if m.sqrt((boylam - cezeri.harita.hastaneler[i].boylam)**2 + (enlem - cezeri.harita.hastaneler[i].enlem)**2) < minimum:
minimum = m.sqrt((boylam - cezeri.harita.hastaneler[i].boylam)**2 + (enlem - cezeri.harita.hastaneler[i].enlem)**2)
enYakin_boy = cezeri.harita.hastaneler[i].boylam
enYakin_en = cezeri.harita.hastaneler[i].enlem
olculdu = True
	Araç hareket halinde olduğu süre boyunca anlık olarak en yakın hastane koordinatlarının enlem ve boylamlarını hesaplayarak' enYakin_boy' ve 'enYakin_en' değişkenlerine atamaktadır.
	
	Döngü boyunca her bir hastane uzaklığı 'minimum' değişkeniyle karşılaştırılmaktadır. Bu uzaklık 'minimum' değerinden küçük ise yeni hastane uzaklığı değeri 'minimum' değişkenine atanmaktadır.

if cezeri.acil_durum:
rota_takip(boylam, enlem, enYakin_boy, enYakin_en)
else:
rota_takip(boylam, enlem, varis_boy, varis_en)
	Eğer ki seyir esnasında herhangi bir acil durum meydana gelirse en yakın hastane koordinatları hedef nokta olarak algılamaktadır. Yoksa normal rotasına devam etmektedir.

6. SENARYO 10: GNSS ARIZASI YAŞANDIĞINDA İNİŞİ TAMAMLAYAN ALGORİTMA

6.1. GNSS Arızasının Tespit Edildiği Veri Yapısı

"cezeri.gnss.hata'' veri yapısından GNSS arıza tespiti yapılmıştır. Örneğin bu yapının 'true' çevirmesi GNSS arızasını göstermektedir.

6.2. GNSS Arıza Sonrası Hava Aracının İrtifasının Hesapladığı Algoritma

ALGORİTMA	AÇIKLAMA

If (not m.isinf(cezeri.lidar.mesafe)) and (not cezeri.lidar.hata):
if cezeri.lidar.mesafe < 30:
cezeri.asagi_git(cezeri.yavas)
 	
elif (not m.isinf(cezeri.radar.mesafe)) and (not cezeri.radar.hata):
if cezeri.radar.mesafe < 30:
cezeri.asagi_git(cezeri.yavas)
else:
cezeri.asagi_git(cezeri.hizli)
	Hedef noktaya iniş esnasında araçta GNSS veri kaybı yaşanırsa, araç hedef noktayla arasındaki anlık yükseklik verilerini lidar sensörünü kullanarak hesaplayıp inişini gerçekleştirmektedir.
	
	Eğer ki lidar sensöründe de arıza meydana gelirse yüksekliği radar sensörü ile hesaplayarak iniş yapmaktadır.

7. SENARYO 11: GNSS ARIZASI YAŞANDIĞINDA SEYİR YAPMAYA DEVAM EDEN VE GÖREVİ TAMAMLAMAYI SAĞLAYAN ALGORİTMA

7.1. GNSS Arızası Yaşandıktan Sonra Hava Aracının (Yatay) Pozisyonunun Hesaplandığı Algoritma

ALGORİTMA	AÇIKLAMA
dondu = False	Aracın B noktasından C noktasına gitmek için yöneliminin henüz tamamlanmadığını gösterir.
vardi_B = False	B noktasına ulaşmadığını gösterir.
vardi_C = False	C noktasına ulaşmadığını gösterir.
gnss_hata = False	GNSS sisteminde arıza olmadığını gösterir
mesafe_x = 0 mesafe_y =0	Aracın GNSS arızasından sonraki konumunun x ve y koordinatlarını belirten değişkenler.
varis0_boylam = cezeri.hedefler[0].boylam varis0_enlem = cezeri.hedefler[0].enlem varis1_boylam = cezeri.hedefler[1].boylam varis1_enlem = cezeri.hedefler[1].enlem	Sırasıyla hedef B ve hedef C noktalarının koordinatlarını saklayan değişkendir.
dt= cezeri.ros_zamani() - son_olcum	İki veri arasında geçen süreyi gösterir.
if 0 <= gnss_boylam[-1] < 0.001 and not gnss_hata: enlem0 = gnss_enlem[-2] boylam0 = gnss_boylam[-2] gnss_hata = True	GNSS'in arızası yaşanmadan önceki verilere ulaşılarak hata olup olmadığı tespit edilmektedir.

if cezeri.gnss.hata and gnss_hata: mesafe_x = mesafe_x + cezeri.imu.Vx*dt mesafe_y = mesafe_y + cezeri.imu.Vy*dt x = round(enlem0 + mesafe_x/20,2) y = round(boylam0 +mesafe_y/20,2) enlem = x boylam = y	Aracın GNSS arızası yaşandıktan sonra başlangıç noktasına göre konumu hesaplan-lanmaktadır.

8. SENARYO 12: GNSS ARIZA DURUMUNU TESPIT EDEN ALGORITMA

8.1. GNSS Karıştırma (Jamming) Ve Yanıltma (Spoofing) Açıklaması ve Farkları

Kablosuz sinyalle çalışan cihazlar, iletişim yöntemi olarak radyo frekansını kullanır. Jamming diğer adıyla frekans bozma veya GNSS karıştırma, kablosuz sinyallerin daha güçlü bir sinyalle bastırılmasıdır. GNSS kısa dalga boylu radyo dalgalarını kullanır. Bu nedenle uçan arabada jamming saldırısıyla daha güçlü sinyallerle bastırılıp kullandığımız GNSS sisteminin karışmasına ve eldeki verilerin hatalı çıkmasına sebebiyet verir.

Spoofing (sahte sinyal-GNSS yanıltma) başkası tarafından kullanıcının sistemine ve bilgilerine kullanıcının kendisiymiş gibi erişebilmesidir. Böylelikle uçan arabadaki GNSS sistemlerindeki verileri yanlış verilerle değiştirerek hatalı konum bilgileri elde edilir.[17–20]

,

8.2. GNSS Yanıltma (Spoofing) Durumunu Tespit Eden Algoritma

ALGORİTMA	AÇIKLAMA
gnss_yaniltma = False	GNNS yanıltma durumunun gerçekleşmediğini gösteren değişkendir.

if len(gnss_boylam) > 10:
if (abs(gnss_boylam[-1] - gnss_boylam[-2]) > 2) and (not gnss_yaniltma):
enlem0 = gnss_enlem[-2]
boylam0 = gnss_boylam[-2]
 	
gnss_yaniltma = True
	GNSS yanıltma durumu gerçekleşmeden önceki 2 veriyi karşılaştırarak arıza tespiti yapmaktadır.

if gnss_yaniltma: mesafe_x = mesafe_x + cezeri.imu.Vx*dt mesafe_y = mesafe_y + cezeri.imu.Vy*dt x = round(enlem0 + mesafe_x/20, 2) y = round(boylam0 +mesafe_y/20,2) enlem = x boylam = y	Yanıltma durumu gerçekleştiğinin tespiti yapıldıktan sonra aracın başlangıç noktasına göre konumunu hesaplamaktadır.

9. SENARYO 13: İNİŞİ TAMAMLAMAK İÇİN KULLANILAN SENSÖR SEÇİMİ İÇİN YAPILAN ALGORİTMA

9.1. İniş İçin Seçilen İrtifa Kaynağı Sensörü

Sensör arızaları gerçekleştikten sonra IMU (Inertial Measurement Unit) sensörü devreye girmiştir. Bu sensör aracın açısal hızını kullanarak konum ve yükseklik hesabı yapmaktadır.

9.2. Diğer Sensörlerin Seçilmeme Sebepleri

Diğer sensörlerin özellikleri incelendi ve aralarında en doğru veriyi veren sensörün IMU olduğuna karar verildi. Çünkü IMU sensörü 3 bileşenden oluşmaktadır ve 3 eksende de hesaplamalar yapmaktadır. Diğer sensörlerin yıpranma payları, titreşim ve hassasiyetleri de hesaba katılarak karşılaştırılmalar yapıldı.

Ayrıca jiroskop ve ivmeölçer gibi sensörler bulunup bu sensörler yeterli bilgiyi sağlamadığı için birleştirilerek IMU sensörü elde edilmiştir.[21,22]

9.3. Bu Seçimi Gerçekleştiren Algoritma

ALGORİTMA	AÇIKLAMA
yukseklik = 0 yukseklik = yukseklik + cezeri.imu.Vz*dt	A noktasının ilk yüksekliğidir ve dinamik olarak değişmektedir.
B_yukselti = cezeri.harita.bolge(varis.enlem, varis.boylam).yukselti	Hedef nokta olan B'nin yüksekliğini veren değişkendir.
A_yukselti = cezeri.harita.bolge(enlem, boylam).yukselti	A noktasının yükseklini veren değişkendir.

if cezeri.radar.hata and cezeri.barometre.hata and cezeri.gnss.hata:
if yukseklik + A_yukselti - B_yukselti < 40:
cezeri.asagi_git(cezeri.yavas)
else:
cezeri.asagi_git(cezeri.hizli)
	Radar, barometer ve GNSS' te arıza meydana geldiği anlaşıldığında iniş yapılacak nokta ile araç arasındaki yükseklik hesaplanarak iniş gerçekleşmektedir.

10. SENARYO 15: MANYETOMETRE VERİ KAYBI YAŞANDIĞINDA GİDİLMESİ GEREKEN HEDEFE HAVA ARACININ YÖNELMESİNİ SAĞLAYAN ALGORİTMA

10.1. Manyetometre Veri Kaybının Tespiti

'manyetometre_hata' Boolean (True-False) yapısı olarak elde edilir. Eğer bu yapı 'True' değeri ise manyetometrede arıza olduğu anlaşılmaktadır.

10.2. Hava Aracının Gitmesi Gereken Yönü Hesaplayan Algoritma

ALGORİTMA	AÇIKLAMA
aci = math.atan2(varis0_boylam - boylam ,varis0_enlem - enlem)*(180/math.pi)	Aracın A noktasından B noktasına gitmesi gereken yönü hesaplayan algoritma gösterilmiştir.
aci = math.atan2(varis1_boylam - boylam ,varis1_enlem - enlem)*(180/math.pi)	Aracın B noktasından C noktasına gitmesi gereken yönü hesaplayan algoritma gösterilmiştir.

10.3. Hava Aracının Gittiği Yönü Hesaplayan Algoritma

AÇIKLAMA	ALGORİTMA
angle=angle +cezeri.imu.acisal_hiz*dt*(180/math.pi)	Manyometrede veri kaybı yaşandığında aracın gittiği yönü hesaplayarak gösteren değişken.

def rota_takip(A_j, A_i, B_j, B_i):
cezeri.ileri_git(cezeri.hizli)
aci = math.atan2(B_j - A_j ,B_i - A_i)*(180/math.pi)
yonelim = cezeri.manyetometre.veri*(180/math.pi)
if 5 > aci - yonelim > 1:
cezeri.don(hiz*faktor)
else:
cezeri.don(-1*hiz*faktor)
	Aracın istenilen hedefe varması için gerekli olan yönü korumak için gerekli olan algoritmadır.

11. SENARYO 16: ŞARJ DURUMUNU GÖZETEREK ŞARJ İSTASYONLARINI TESPİT EDEN VE HAVA ARACINI ORAYA YÖNLENDİREN ALGORİTMA

11.1. Batarya Durumları İçin Kullanılan Veri Yapısı

‘cezeri.batarya.veri’ integer yapısı ile bataryanın anlık durumu elde edilir.

11.2. Şarj İstasyonuna İnilmesi Gereken Batarya Gerilim Eşiği ve Nedenleri

Batarya gerilim eşiği %40 değerine düştüğünde en yakın sarj istasyonuna iniş yapacaktır. Seyir esnasında birden çok kez şarj istasyonuna uğrayıp uçuş süresini uzatmamak için gerilim değeri yüksek tutulmamıştır. Ayrıca şarj istasyonuna ulaşıncaya kadar bataryanın bitmemesi için de düşük bir değer verilmemiştir. Bu yüzden optimum değer %40 olarak belirlenmiştir.

11.3. Şarj İstasyonlarının Olduğu Veri Yapısı

‘cezeri.harita.sarj_istasyonlari’ ile bu veriler list yapısı olarak elde edilmiştir. Örnek olarak ‘cezeri.harita.sarj_istasyonlari[0]’ verisi ile ilk şarj istasyonun konumu elde edilir.

TEKNOFEST
İSTANBUL HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

12. KAYNAKÇA

1. Kodlama İçin En Gerekli 16 Programlama Dili » Kozan Demircan. https://khosann.com/kodlama-icin-en-gerekli-16-programlama-dili/.
2. 2021'in En iyi 20 Yazılım (kodlama) Dilleri Nelerdir? – Geleceğin Seyahat Teknolojileri. https://adviceal.com/teknoloji/2021-en-iyi-20-yazilim-kodlama-dilleri-nelerdir/.
3. Toptalent.co | En Popüler 25 Programlama Dilleri 2021. https://toptalent.co/en-populer-yazilim-ve-programlama-dilleri.
4. Barometre Nasıl Çalışır? » Bilgiustam. https://www.bilgiustam.com/bir-barometre-nasil-calisir/.
5. Basınç Sensörü Nedir? Basınç Sensörü Çeşitleri » Sensortek - HBM. https://www.sensortek-hbm.com/basinc-sensoru-nedir/.
6. barometre basınç sensörü. https://tr.jf-parede.pt/barometric-pressure-sensor-circuit-working.
7. (100) Barometre Nasıl Çalışır ? - YouTube. https://www.youtube.com/watch?v=W4y1hJjUAtY.
8. How Many Different "GNSS" Systems Are There? - Surveying Group. https://surveyinggroup.com/how-many-different-gnss-systems-are-there/.
9. (100) Küresel Seyrüsefer Uydu Sistemleri (GNSS:Global Navigation Satellite System) - YouTube. https://www.youtube.com/watch?v=-mVlXwFmvTQ.
10. (100) GPS Nedir? Navigasyon Cihazları Nasıl Çalışır? GNSS Nedir? - YouTube. https://www.youtube.com/watch?v=n927KqJGOw4.
11. Günhan, Y., Doç, Y. & Demġrer, M. ATALETSEL ÖLÇÜM BĠRĠMĠ HATALARININ BELĠRLENMESĠ VE DÜZELTĠLMESĠ DETECTION AND COMPENSATION OF INERTIAL MEASUREMENT UNIT ERROR.
12. Korul, V. *et al.* Havaalanı Çevre Yönetim Sistemi.
13. uçak iklimlendirme sistemleri. https://www.mmo.org.tr/sites/default/files/6ba0bd323b1609f_ek.pdf.
14. Uçak inişe geçtiğinde dikkat edilmesi gerekenler | ENUYGUN. https://www.enuygun.com/ucak-bileti/sikca-sorulan-sorular/ucak-inise-gectiginde-dikkat-edilmesi-gerekenler/.
15. Tarihi, Y. SHT-İNİŞ ŞERİDİ İNİŞ ŞERİTLERİ İNŞA EDİLMESİ VE KULLANILMASI TALİMATI (SHT-İNİŞ ŞERİDİ).

16. HAVAALANI DIŞINDA İNİŞ BÖLGESİ OPERASYONLARI HELİKOPTER PİLOTLARININ EMNİYETİ İYİLEŞTİRME YÖNTEMLERİ EĞİTİM BROŞÜRÜ.
17. Jamming nedir? Jamming saldırıları nasıl önlenir? - Makale Detayı. https://www.guvenlikonline.com/makale/553/jamming-nedir-jamming-saldirilari-nasil-onlenir.html.
18. Anti-Jamming GNSS Sistemi – Defense Here. https://www.defensehere.com/tr/anti-jamming-gnss-sistemi.
19. (1) Uydularla Konum Belirleme Sistemlerinin Siber Güvenliği | LinkedIn. https://www.linkedin.com/pulse/uydularla-konum-belirleme-sistemlerinin-siber-g%C3%BCvenli%C4%9Fi-nazli-can/?originalSubdomain=tr.
20. Anti-Jamming GNSS Sistemi – Defense Here. https://www.defensehere.com/tr/anti-jamming-gnss-sistemi.
21. IMU Nedir? | CBS AKADEMİ. https://cbsakademi.ibb.istanbul/imu-nedir/.
22. Konum Sensörleri – için Basit Anlatımlı Kılavuz - Zettlex Endüktif Kodlayıcılar. https://www.celeramotion.com/zettlex/tr/destek/teknik-belgeler/konum-sensorleri-icin-basit-anlatimli-kilavuz/.

TEKNOFEST
İSTANBUL HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ